

sc0red Engineering Pipeline v2.0

Version: 2.0 — 2026-04-01 Author: Scarlett, revised by Chris Creel Status: Approved —

Overview

This document defines how engineering work flows from request to production at sc0red. It covers ticket lifecycle, agent responsibilities, human gates, branching strategy, and environment promotion.

Every engineer and agent follows this process. No exceptions.

Related documents: - [Estimation & Prioritization Framework](#) — how we estimate, score, and order work

Roles

Role	Who	Responsibility
Requester	Any user or team member	Submits feature request or bug report
Business Leadership	Chris, Zack, Brian	Prioritizes, sets requirements, moves to Plan
Patch (Agent)	Senior AI Engineer	Investigation, design, implementation, PR creation, environment promotion
Scarlett (Agent)	Senior Reviewer	Plan review, PR code review, architectural oversight
Engineering Team	Srikanth, Vedratna, Srilakshmi	PR review, verification in dev/testing, production readiness
Production Approver	Chris Creel (only)	Approves production deployments

Event-Driven Architecture — Clawndom

All agent work is triggered by **Jira webhooks** via **Clawndom**, a webhook proxy that sits between Jira and OpenClaw. Clawndom:

1. Receives Jira webhooks via Tailscale Funnel
2. Validates HMAC signatures
3. Queues events in Redis (BullMQ)
4. Routes to the correct agent based on the issue’s status name
5. Delivers one event at a time (global serialization via MAX_CONCURRENT_RUNS=1)
6. Waits for the agent session to complete before delivering the next event
7. Retries on failure (max 2 attempts, back-of-queue retry)

Each webhook spawns an **isolated agent session** — Patch’s main thread is never interrupted. Clawndom handles serialization, so agents do not need to implement their own work gates or priority selection. They process whatever Clawndom feeds them, in order.

Routing rules match on `issue.fields.status.name` and render a Nunjucks message template with the Jira payload before delivering to the agent.

Config: PROVIDERS_CONFIG JSON env var in the launchd plist. Templates live at `~/ .openclaw/workspace-patch/templates/`.

Board Columns (Jira Statuses)

Column		Transition ID	Category	Routed to Agent?
Backlog (New)	11		To Do	No
Plan	25		In Progress	Yes — Patch investigates, writes plan
In Planning	23		In Progress	No
Plan Review	26		In Progress	No — human gate
Ready for Development	3		In Progress	Yes — Patch implements
In Development	21		In Progress	No
Code Review	22		In Progress	No — human gate
Deploy to development	8		In Progress	Yes — Patch merges PR
Verified in Development	12		In Progress	Yes — Patch checks gate, promotes to testing
Deploy to testing	10		In Progress	Yes — Patch confirms branch state
Verified in Testing	5		In Progress	Yes — Patch checks gate, transitions to Ready for Production
Ready for Production	7		In Progress	Yes — Patch checks gate, creates production PRs (NO merge)
Deployed to Production	31		Done	No
Hotfix	27		In Progress	Yes — Patch creates PRs against production + back-merges
Blocked	4		To Do	No — human resolves

Ticket Lifecycle

1. New (Backlog)

- **Who creates:** Requester (user, PM, human engineer, agent engineer)
- **What happens:** Ticket exists but hasn't been evaluated
- **Next:** Business leadership reviews and prioritizes

2. Plan

- **Who transitions:** Business leadership
- **Trigger:** Jira webhook → Clawndom → Patch
- **What happens:** Patch immediately picks up the ticket and begins:
 - **Quality gates first** — checks for insufficient info, conflicting info, unclear scope, multiple work items. If any gate fails → Blocked (transition ID: 4) with a comment explaining what's needed.
 - **Investigation** — check logs, database, CloudWatch. Form diagnosis from evidence.
 - Root cause analysis (bugs) or requirements analysis (features)
 - Risk assessment and severity estimation per the [Estimation & Prioritization Framework](#)
 - Story point lookup from the Risk × Intensity matrix
 - If SP > 5: must propose a breakdown before implementation
 - Design proposal with affected files, line numbers, and approach
- **Design quality requirements:**
 - Prefer structural improvements over quick fixes. If the underlying architecture is wrong, say so.
 - Apply established design patterns (GoF, architectural patterns) where appropriate — don't invent when a known pattern fits.

- Consider downstream consequences — will this fix create the next bug? Does this component need extraction, not patching?
- Small files, typed interfaces, consistent patterns. If you're adding to a god file, flag it.
- A hack that ships is still a hack. Propose the right fix; if time/risk forces a compromise, document the tech debt explicitly.
- **Review request:** Patch spawns a Scarlett subagent for review. Scarlett reviews and sends verdict via `sessions_send`.
- **Transition:** Patch posts plan as Jira comment, updates custom fields (Risk, Intensity, Story Points, Velocity Impact), transitions to **Plan Review** (ID: 26).

3. Plan Review

- **Who reviews:** Scarlett (agent) + Engineering team (human)
- **Scarlett checks:**
 - Is this solving the right problem? Root cause or symptom?
 - Does the design use appropriate patterns, or is it a hack?
 - Structural/architectural implications — god files, tight coupling, missing abstractions?
 - Would a refactor serve better than a patch?
 - Edge cases, downstream consequences, cross-component impact
 - Is the estimation reasonable given the risk and severity?
- **Human review:** Engineers review Patch's plan (already vetted by Scarlett). May refine or approve as-is.
- **Next:** Engineer moves ticket to **Ready for Development**

4. Ready for Development

- **Who transitions:** Engineering team
- **Trigger:** Jira webhook → Clawndom → Patch
- **What happens:** Clawndom serializes delivery — Patch processes one ticket at a time, in queue order. No priority selection or work gates needed; Clawndom handles sequencing.
- **Patch's process:**
 1. **Immediately** transitions to **In Development** (ID: 21) — board reflects reality before any work starts
 2. Gets Jira OAuth token
 3. Reads the approved plan from Jira comments
 4. Creates branch: `fix/<jira-key>--<short-slug>` off development
 5. Spawns Claude Code to implement exactly the approved plan
 6. Reviews output — diff matches plan, tests exist, no scope creep
 7. Opens PR against development
 8. Transitions to **Code Review** (ID: 22)
 9. Posts PR link as Jira comment
 10. Spawns Scarlett subagent for PR review

5. In Development

- **Scarlett reviews the PR:**
 - Correctness — does it match the approved plan?
 - Design quality — follows established patterns? Appropriate abstractions?
 - Consistency — follows existing conventions in the codebase?
 - Edge cases — stale refs, race conditions, missing null checks?
 - Tests — adequate coverage?
- **How:** Scarlett posts review comments on the **GitHub PR** (permanent record for humans), then sends verdict via `sessions_send`.
- **Back-and-forth:** Patch addresses feedback, pushes updates, spawns another Scarlett review. Repeat until clean.
- Once Scarlett approves → Patch transitions to **Code Review** (ID: 22)

6. Code Review

- **Who transitions here:** Patch (after Scarlett approves)
- **What it means:** PRs are open, Scarlett has approved, ready for human engineer review.

- **Jira comment:** Patch posts a consolidated comment listing every open PR for this ticket.
- **Human PR Review:**
 - Chris reviews from Jira
 - Focus on: business logic correctness, UX implications, judgment calls
 - **If changes needed:** Move ticket back to **Ready for Development** (ID: 3). Patch picks it up, reworks, and cycles back through.
 - **If approved:** Move ticket to **Deploy to development** (ID: 8).

7. Deploy to development

- **Who transitions:** Engineer (after PR approval)
- **Trigger:** Jira webhook → Clawndom → Patch
- **What happens:**
 1. Patch finds the PR and merges it to development (squash + delete branch)
 2. Confirms CI passed
 3. Posts Jira comment confirming merge
 4. If CI fails → Blocked (transition ID: 4)
- **Next:** Engineers verify the fix works in the dev environment, then move to **Verified in Development**

8. Verified in Development

- **Who transitions:** Engineer (after verifying fix works in dev)
- **Trigger:** Jira webhook → Clawndom → Patch
- **Gate logic:** Patch checks if ANY tickets remain in “Deploy to development”
 - **If yes:** Posts comment listing what’s still awaiting verification. Stops.
 - **If no (all verified):** Proceeds to promote.
- **Promotion:**
 1. Collects all tickets in “Verified in Development”
 2. Creates and **merges** development → testing PRs across all 3 repos (Frontend, Backend, Engine)
 3. Transitions all verified tickets to **Deploy to testing** (ID: 10)

9. Deploy to testing

- **Who transitions:** Patch (during promotion)
- **Trigger:** Jira webhook → Clawndom → Patch
- **What happens:** Patch confirms testing branch is up to date with development
 - If behind → Blocked (transition ID: 4)
 - If current → Posts confirmation comment
- **Next:** Engineers verify in test environment, then move to **Verified in Testing**

10. Verified in Testing

- **Who transitions:** Engineer (after verifying fix works in test)
- **Trigger:** Jira webhook → Clawndom → Patch
- **Gate logic:** Patch checks if ANY tickets remain in “Deploy to testing”
 - **If yes:** Posts comment listing what’s still awaiting verification. Stops.
 - **If no (all verified):** Transitions all verified tickets to **Ready for Production** (ID: 7)

11. Ready for Production

- **Who transitions:** Patch (from Verified in Testing gate) or Engineer
- **Trigger:** Jira webhook → Clawndom → Patch
- **Gate logic:** Patch checks if ANY tickets remain in “Verified in Testing”
 - **If yes:** Posts comment. Stops.
 - **If no (all cleared):** Creates testing → production PRs across all 3 repos. **Does NOT merge.**
- **Next:** Chris reviews and merges the production PRs, then moves tickets to **Deployed to Production**

12. Deployed to Production

- **Who transitions:** Chris (after merging production PRs)
 - **What happens:** Done. Ticket closed.
-

Hotfix Flow

For critical fixes that must reach production immediately, bypassing the normal promotion pipeline.

Process

1. Ticket goes through Plan → Plan Review → Ready for Development → Code Review as normal
2. Instead of Deploy to development, engineer moves ticket to **Hotfix** (ID: 27)
3. **Trigger:** Jira webhook → Clawndom → Patch
4. **Patch’s process:**
 - Finds the feature branch
 - Rebases onto production (not development)
 - Creates PR against production — titled HOTFIX(<key>): <summary>
 - Creates back-merge PRs: feature branch → testing and feature branch → development
 - Spawns Scarlett for urgent review
 - Posts all 3 PR links to Jira
 - **Does NOT merge** — human merges the production PR
5. After human merges to production, human merges the back-merge PRs and moves ticket to **Deployed to Production**

Branching

- Normal flow: feature branch based on development
 - Hotfix flow: feature branch **rebased onto production**, PRs target production + back-merge to testing and development
 - This ensures the fix reaches production without dragging unverified development work with it
-

Blocked

“Blocked” (transition ID: 4) is a universal status used across the entire pipeline when: - Ticket has insufficient information, conflicting requirements, or unclear scope - CI fails on a PR - Testing branch is out of sync - No approved plan exists - Any situation requiring human intervention

Who moves to Blocked: Patch (any template) or any human **Who resolves:** The original reporter or an engineer. After resolving, they move the ticket back to the appropriate column (Plan, Ready for Development, etc.), which re-triggers the Clawndom webhook.

Promotion Protocol — Gate-Driven Branch Merges

Individual tickets merge to development via per-ticket PRs. Promotions between environments are **gate-driven** — triggered automatically when all tickets in a deployment column are verified.

Development → Testing

- **Trigger:** Last ticket moves from “Deploy to development” to “Verified in Development”
- **Who executes:** Patch (automatic via Clawndom webhook)
- **Process:**
 1. Patch verifies “Deploy to development” is empty

2. Creates development → testing PRs across all 3 repos
3. **Merges** the PRs
4. Transitions all “Verified in Development” tickets to “Deploy to testing”

Testing → Production

- **Trigger:** Last ticket moves to “Ready for Production” and “Verified in Testing” is empty
- **Who executes:** Patch creates PRs, **Chris merges**
- **Process:**
 1. Patch verifies “Verified in Testing” is empty
 2. Creates testing → production PRs across all 3 repos
 3. Posts PR links to Jira
 4. **Does NOT merge** — Chris reviews and merges

Rules

- Development → testing: Patch can merge (automated gate)
- Testing → production: Patch creates PR only, human merges (production gate stays human)
- If CI fails on a promotion PR, Patch reports immediately and does not merge
- Promotions are all-or-nothing per branch merge. If a specific ticket needs to be held back, it should not be merged to development until it’s ready to travel with the batch.

Build Validation (by repo)

Before opening a PR, the build must not be broken. What that means varies:

Repo	Validation Command	What It Checks
Platform-Frontend	tsc --noEmit + test suite	Type safety + unit tests
Platform-Backend	pytest	Unit + integration tests
assessment_engine	make check-all	Full lint, type check, and test suite

If the command fails on code you didn’t touch, **that’s still our problem**. File a ticket immediately, fix it if you can, and note it in your PR. Nothing in this codebase is someone else’s job.

Agent Communication — How Engineering Agents Talk

All structured agent-to-agent communication for code review happens via `sessions_send` (spawned subagents):

Review Flow (`sessions_send`)

- **Plan review requests:** Patch spawns a Scarlett subagent with Jira link + summary. Scarlett sends verdict via `sessions_send` to `agent:patch:main`.
- **PR review requests:** Patch spawns a Scarlett subagent with PR link + ticket reference. Scarlett reviews on GitHub and sends verdict via `sessions_send`.
- **Back-and-forth:** Patch fixes issues, spawns another Scarlett review. No passive monitoring needed — delivery is guaranteed.

GitHub PR Comments

- **Code review feedback:** Scarlett posts line-level and summary comments directly on the GitHub PR via `gh`
- **Patch responses:** Patch reads PR comments, pushes fixes, spawns another review
- **Why GitHub:** This is where human engineers will also review. Keeping all code-level discussion on the PR means one place for everyone.

What goes where

- Review requests → spawned Scarlett subagent
- Code-level feedback → GitHub PR comments + verdict via sessions_send
- PR approved, requesting human review → #general-engineering (Slack)
- Blocked on something → #general-engineering (Slack)

Branching Strategy

Branch	Environment	Purpose
development	Development	Active development, all feature branches merge here
testing	Testing	Verified dev changes promoted for stakeholder testing
production	Production	Release branch, Chris-only merge approval
• Feature/fix branches: fix/<jira-key>-<short-slug> off development • Hotfix branches: fix/<jira-key>-<slug> rebased onto production, with back-merge PRs to testing and development • No direct commits to testing or production — always via PR		

Communication Channels

Channel	Platform	ID	Purpose
#general-engineering	Slack	C06TRR7A894	PR review requests to humans, blocked tickets, production readiness pings, deploy confirmations
#general-engineering-qa	Slack	C0ALJS0M2NR	QA findings, test results, plan summaries
#alerts-platform-failure-*	Slack	Various	Automated error alerts (dev/testing/production)
#general	Discord (The Agency)	1478849414629032154	Cross-agent coordination, non-review discussion
Jira comments	Jira	—	Plan proposals, review feedback, permanent technical record
GitHub PR comments	GitHub	—	Code-level review (Scarlett + engineers)

Escalation Rules

Patch escalates to Chris (moves to Blocked or flags in Slack) when: - Fix touches auth or security - Root cause is in the backend API contract - Estimated risk is High - She disagrees with a human reviewer’s feedback and can’t resolve it - CI fails for reasons outside her change

Implementation Tooling — Claude Code

Agents use **Claude Code** (via `sessions_spawn` with `runtime: "acp"`) as the implementation engine. Agents don't edit files directly for multi-file changes — they spawn Claude Code sessions with full repo context, the approved plan, and coding standards.

Why Claude Code

- Loads full repo into context — file trees, type definitions, imports, test suites
- Makes coherent multi-file changes that respect existing patterns
- Runs in-process validation (type checking, linting)

Session Lifecycle

1. Agent receives work from Clawndom (ticket in the appropriate status)
2. Agent spawns Claude Code with: repo path, branch, task description, constraints
3. Claude Code executes — writes code, runs checks
4. Agent reviews output — commits, test results, diff quality
5. Session terminates — RAM freed

Local Validation — MANDATORY Before Push

Never push code that hasn't been validated locally. CI is not your first line of defense — it's your last.

Repo	Command	What to check
Platform-Frontend	<code>npx ng test --watch=false + npx tsc --noEmit</code>	Unit tests + type check
Platform-Backend	<code>pytest path/to/changed/tests/</code>	Tests for changed modules
assessment_engine	<code>make check-all</code>	Full lint, type check, and test suite

Rules: - Type check: **every push, no exceptions** - Unit tests for changed files: **every push, no exceptions** - If local validation fails on code you didn't touch: file a ticket, note it in the PR, fix if you can - **Do not rely on CI to catch problems you could have caught locally in 30 seconds**

Key Constraints

- Clawndom serializes all agent work — one event at a time, completion-aware
- No implementation without an approved plan
- No merge without CI passing
- No production merge without Chris's explicit approval (Patch creates PR only)
- Development → testing promotion: Patch can merge (automated gate)
- Testing → production promotion: Patch creates PR, human merges
- A fix without unit tests is not done
- Prefer structural improvements over quick hacks — if the architecture is wrong, say so
- Refactoring is a valid fix — but requires documented architectural justification and approval